

École Polytechnique Fédérale de Lausanne

Termination Checking in Scala

by Ozair Faizan

Master Semester Project Report

Approved by the Examining Committee:

Thesis Advisor:
Viktor Kunčak

PhD Student Supervisor:
Matthieu Bovel

05 June 2026

Abstract

Termination is a crucial property for safety-critical systems and plays an important role in proof assistants, where restrictions to terminating definitions can be necessary to preserve soundness. While most proof assistants enforce termination strictly, they are often too strict for general-purpose programming. Existing verification tools for Scala, such as Stainless, rely on external SMT solvers that add significant verification time and dependencies. This leaves a gap for a built-in, lightweight mechanism to verify termination natively without an external framework.

This report presents the design and implementation of an opt-in termination checker for the Scala 3 compiler. Methods annotated with `@terminates` are verified to be terminating by leveraging structural decrease analysis of Algebraic Data Types (ADTs).

The checker can handle termination of recursive functions, mutual recursion and while loops, but inevitably flags some terminating functions. To maintain usability, developers can utilize escape hatches to manually assert that some functions terminate or that some ADTs are well-founded. If no errors were raised, the annotated method is guaranteed provided that all such assertions are valid.

Contents

Abstract **2**

1 Introduction **5**

2 Background and Related Work **8**
2.1 Termination Analysis Techniques 8
2.1.1 Lexicographic Ordering 8
2.1.2 Size-Change Principle 8
2.1.3 User-Provided Proofs 9
2.2 Existing Tools 9
2.2.1 Rocq, Agda, and Lean 9
2.2.2 AProVE 9
2.2.3 Stainless 10

3 Design and Approach **11**
3.1 Conservative, Sound Analysis 11
3.2 Escape Hatches 12

4 Implementation **14**
4.1 Structural Decrease Analysis 14
4.1.1 Pattern Matching and ADTs 14
4.1.2 Field Access and `peelSelects` 15
4.1.3 Verifying Polymorphic Method Calls 15
4.1.4 Well-Foundedness 15
4.1.5 Lexicographic Ordering 16
4.2 Mutual Recursion 16
4.3 While Loops 16
4.3.1 `varMap` Updates 17

5 Evaluation and Limitations **18**
5.1 Performance Evaluation 18
5.2 What the Checker Can Verify 19

5.2.1 Simple Structural Recursion . . . 19
5.2.2 Lexicographic Decrease and `@decreases` . . . 19
5.2.3 Mutual Recursion . . . 20
5.2.4 Higher-Order Functions . . . 20
5.2.5 While Loops . . . 21
5.3 What the Checker Cannot Verify . . . 22
5.3.1 Semantic Termination Arguments . . . 22
5.3.2 Cross-Parameter Size Tracking . . . 23

6 Correctness **24**
6.1 Simple Recursion . . . 24
6.2 Mutual Recursion . . . 25
6.3 While Loops . . . 25
6.4 Higher-Order Functions . . . 26
6.5 Soundness Under Escape Hatches . . . 26

7 Discussion **27**
7.1 Annotations Added to the Standard Library . . . 27
7.2 Lexicographic Ordering over the Size-Change Principle . . . 28
7.3 Limitations . . . 28

8 Conclusion **29**

Bibliography **30**

Chapter 1

Introduction

Termination is a fundamental property of computer programs and a cornerstone of formal verification. Infinite loops can lead to system failure, resource exhaustion, and application crashes. However, ensuring that a program will eventually halt, known as the Halting Problem, was proved to be undecidable in the general case by Alan Turing [Tur37]. Since no algorithm can universally solve the Halting Problem, we must rely on restricted techniques that are sound however not complete.

When working with total functions, termination is a requirement for logical consistency in formal verification and proof assistants. Indeed, in such settings, non-terminating functions can be exploited to effectively allow the type system to accept proofs of false statements [TF25]. To illustrate this vulnerability in a system utilizing refinement types such as [BKO26], consider how an infinite loop can trick the compiler into accepting a logical contradiction by forging an impossible type:

```
def asNonEmpty[A](l: List[A]): {b: Boolean with b == true && l != Nil} =
  val res = asNonEmpty(l)
  res
@main def main =
  val l = Nil: {l: List[Int] with asNonEmpty(l)}
```

The non-terminating function `asNonEmpty` is considered well-typed, and therefore the type checker accepts the impossible refinement type for `Nil`. While the type system's logical soundness is compromised at compile time, the impossible type is not materialized at runtime.

It is worth noting that there are consistent formal systems that do not require termination to derive correct conclusions. They are usually based on partial functions. Notable examples

include the domain theory as mechanized in the LCF proof assistant family [PNW19], and more recently, the Iris framework [JUN+18]. However, we want to work with total functions over algebraic data types, as such termination remains a concern.

Consequently, proof assistants dealing with total functions such as Rocq, Agda, and Lean employ techniques such as recursive calls to be on syntactically smaller arguments, size-change principle [LJB01], well-founded measures [DM79], or manually provided proofs. For example, in a proof assistant like Rocq, termination is established when recurring on structurally smaller components of an inductively defined type:

```
Fixpoint sum (l: list nat): nat :=
  match l with
  | [] => 0
  | hd :: tl => hd + sum tl
  end.
```

However, proof assistants are often too rigid for general-purpose programming. Tools like Stainless [HVK19] provide a more flexible approach by leveraging powerful SMT (Satisfiability Modulo Theories) solvers, capable of proving complex termination properties. Stainless excels at verifying semantic termination bounds over bounded and unbound integers. For instance, Stainless can prove the termination of

```
def sum(l: List[BigInt]): BigInt =
  if l.isEmpty then 0
  else l.head + sum(l.tail)
```

But Stainless relies on external tools, and operates as an external toolchain to Scala. This leaves a gap in general-purpose languages where developers often lack a built-in, lightweight mechanism to verify the termination of critical functions without relying on a separate verification framework.

To bridge this gap, this report presents an opt-in termination checker implemented natively as a `MiniPhase` within the Scala 3 compiler. It operates in a middle ground between the heavyweight, strict checks of proof assistants and the lack of it in general-purpose languages. Similar to proof assistants, it ensures termination natively through structural decrease analysis over Algebraic Data Types:

```scala
@terminates
def sum(list: List[Int]): Int =
  list match
    case Nil => 0
    case hd :: tl => sum(tl)
```

The implementation focuses on common patterns Scala developers encounter, however due to the lightweight structural analysis, it inevitably flags some complex terminating functions as possibly non-terminating. To maintain usability, we provide several escape hatches. Developers can use annotations like `@assumeTerminates`, `@assumeDecreases`, and `@assumeWellFounded` to manually assert properties that the checker cannot automatically infer. This approach provides a native and scalable way to enforce termination, falling back on developers assertions when the automatic analysis limits are reached.

Chapter 2

Background and Related Work

2.1 Termination Analysis Techniques

The theory of termination analysis relies on the notion of well-founded relations. A binary relation $\prec$ on a set S is well-founded if there are no infinite descending chains. That is, there exists no infinite sequence $(s_n)_{n\in\mathbb{N}}$ of elements in S such that $s_{n+1} \prec s_n$ for all $n \in \mathbb{N}$ [Win93, p. 31]. If every recursive call can be shown to pass arguments that are strictly smaller than the parameters for some well-founded relation, the function terminates. The following techniques, commonly used in existing tools, extend this idea.

2.1.1 Lexicographic Ordering

In functions with multiple arguments, we can construct a well-founded relation for the tuple of arguments from well-founded relations on each argument. Formally, given well-founded relations $\prec_i$ on sets A_i, the lexicographic order $\prec_{\text{lex}}$ on $A_1 \times \cdots \times A_n$ is defined as [Win93, p. 175]:

$$(a_1, \ldots, a_n) \prec_{\text{lex}} (b_1, \ldots, b_n) \iff \exists i \in \{1, \ldots, n\} \text{ such that } a_i \prec_i b_i \text{ and } \forall j < i, a_j = b_j.$$

2.1.2 Size-Change Principle

The size-change principle [LJB01] is a global termination analysis techniques that models how the size of values change at each function call. The analysis abstracts a program into a finite set of *size-change graphs*, where each graph represents a function call and captures the size relations between the caller's and callee's parameters. Size change termination is specified as

a decision problem: given a finite set of size-change graphs, does every infinite sequence of calls contain an infinite decreasing thread of parameters? If the answer is yes, the program must terminate, as well-founded data cannot decrease indefinitely. Although quite effective, the algorithm to solve this decision problem requires computing the transitive closure of the graphs which is computationally expensive and has been shown to be PSPACE-complete.

2.1.3 User-Provided Proofs

When automatic analysis fails, systems fall back on human intervention. In proof assistant environment, these are mathematical proofs. In the context of this work, this technique takes the form of escape hatches.

2.2 Existing Tools

The challenge of verifying program termination has led to the development of various tools, ranging from proof assistants to automated analyzers.

2.2.1 Rocq, Agda, and Lean

Rocq [Tea25], Agda [Nor07], and Lean [MU21] are state-of-the-art interactive proof assistants based on dependent type theory. They enforce termination for all functions by default. Rocq uses a syntactic guard condition for fixpoints, Agda and Lean employ a more sophisticated termination checker based on the above techniques. They all fall back on manually provided proofs, which is often too labor-intensive for general-purpose software engineering.

2.2.2 AProVE

AProVE [EHG25] is an automated system for proving termination for general-purpose languages such as Java, C, and Haskell. It has achieved state-of-the-art results in the Termination Competition [1]. For Haskell, [Gie+11] implements in AProVE an approach, that symbolically evaluates a given start term under Haskell's lazy evaluation to construct a termination graph, then translates that graph into dependency-pair problems solved by term rewriting techniques.

[1] `https://termination-portal.org/wiki/Termination_Competition_2025`

2.2.3 Stainless

Stainless [HVK19] is a verification framework specifically designed for Scala. It translates Scala code into logical statements, verified by an underlying SMT solver. Stainless can automatically prove intricate mathematical bounds on integers, infer well-founded relations to use, and has semantic awareness of values. However, Stainless operates as a separate toolchain, requires external dependencies and adds significant verification time.

Chapter 3

Design and Approach

The checker is implemented entirely within the Scala 3 compiler as a `MiniPhase`, operating directly on the typed abstract syntax tree (AST) produced by earlier compiler phases. It does not rely on any external solver, rewriting engine, or verification framework. All information needed is obtained through the compiler's own symbol and type APIs. This makes the checker lightweight, and immediately available to any project that uses the Scala 3 compiler, without additional installation or configuration.

Furthermore, unlike proof assistants that enforce termination globally, this checker is entirely opt-in. This aligns with the needs of a general-purpose language like Scala, where the overhead of proving the termination of every method is impractical. The phase is enabled by the `-Ycheck-termination` compiler flag and only methods annotated with `@terminates` and methods called by the annotated methods are checked. Additionally, the phase requires the bodies of functions for the analysis in order to inline function calls, as such, the `-Yretain-trees` flag is required to retain the ASTs of the definitions' bodies, since otherwise only signatures of definitions are kept.

Termination contract can be enabled at the interface level. The checker enforces that all overriding methods inherit the `@terminates` annotation. If an overriding method omits it, an error is thrown, ensuring that termination intents stated at the interface level are maintained.

3.1 Conservative, Sound Analysis

The checker prioritizes soundness over completeness: it will never accept non-terminating methods, however, it may reject methods that do in fact terminate. During the AST traversal, the checker assigns every value a `Size` (either `Smaller`, `Same`, or `Unknown`) relative to the

parameters of the method, then, at each recursive call, the arguments to the call are compared lexicographically to the method's parameters. However, the sizes are inferred syntactically, meaning that we only track a decreasing size when selecting fields of ADTs. Because other measures such as those that require reasoning about arithmetic are out of scope, the checker will reject programs that terminate if the termination relies on such semantic properties.

3.2 Escape Hatches

Because the analysis is conservative, rather than forcing developers to restructure their code for the benefit of the checker, escape hatches are provided as annotations, shifting the analysis on the developer. Misuse of these escape hatches can cause the checker to accept a non-terminating function, so they should be used with care.

The `@assumeTerminates` annotation applied to a method causes the checker to accept any call to that method without inspecting its body. This is especially useful for Scala's standard library methods whose bodies are unavailable. For instance, primitive operators like addition terminate but their body is not available for analysis:

```
@assumeTerminates
def +(x: Int): Int
```

The `uncheckedTermination` method allows the user to wrap a block that is not checked, this is useful for wrapping calls to external libraries when the source is unavailable to the user. Here we bypass the termination check for an external function call while keeping the rest of the recursive method verified:

```
@terminates
def sum_f(l: List[Int]): Int =
  l match
    case Nil => 0
    case hd :: tl => uncheckedTermination(external_f(hd))) + sum_f(tl)
```

The `@assumeDecreases` annotation applied to a method (typically a field accessor or a selector-like method), tells the checker that the called method on a value outputs a result that is strictly smaller than that value. This is useful when it's semantically clear that the value decreases, but the simple structural analysis fails. In this example, the annotation asserts that `getLeftOrElseRight` yields a structurally smaller value:

```
sealed trait Tree:
  @assumeDecreases
  def getLeftOrElseRigth: Tree = this match
      case Node(left: Node, _) => left
      case Node(Leaf, right) => right
      case _ => Leaf

case class Node(left: Tree, right: Tree) extends Tree
case object Leaf extends Tree
```

The `@assumeWellFounded` annotation applied to a type bypasses the well-foundedness check, asserting that the type admits no infinite descending chains. It is needed for types that are structurally well-founded but whose shape does not match the simple syntactic criterion the checker applies. This allows us to mark the standard library `List` as well-founded regardless of the implementation details:

```
@assumeWellFounded
sealed abstract class List[+A]
```

Chapter 4

Implementation

The checker is implemented as a `MiniPhase` inside the `dotty.tools.dotc.transform` package. A `MiniPhase` is a compiler phase designed so that several mini phases can be merged into a single AST traversal for efficiency [PLO17]. When a method is annotated with `@terminates`, is not abstract (`Deferred`), and has not already been verified in a prior traversal, the methods body is traversed. Successfully checked methods are accumulated in a `checked` set shared across all invocations of the phase. A method that has already been certified is not re-examined when it is called by another annotated method. The phase is entirely inactive unless the `-Ycheck-termination` compiler flag is set, ensuring zero overhead for projects that do not opt in.

4.1 Structural Decrease Analysis

The core analysis tracks how the sizes of values evolve as execution proceeds. A `valMap` associates local value symbols to a (*source*, *size*) pair, recording where the value came from and whether it is `Smaller` or `Same` relative to that source. An analogous `varMap` tracks mutable variables, updated at every assignment.

4.1.1 Pattern Matching and ADTs

Pattern matching on an Algebraic Data Type is the primary mechanism by which structural decrease is established. When the selector of a `match` has a synthetic `unapply` method, i.e., one generated automatically by the compiler for a `case class` or `enum`, the checker knows that the binders introduced by each pattern are strict sub-value of the selector. The method

`mapMatchedSymbols` walks the pattern tree. The outermost binder (the variable that names the whole matched value) is recorded as `Same` size as the selector, while any nested binders (the fields extracted from the constructor) are recorded as `Smaller`. If the `unapply` method is user-defined (not synthetic), the checker cannot guarantee that deconstruction actually yields a smaller value, and emits a warning, and does not consider the sub-value as `Smaller`.

4.1.2 Field Access and `peelSelects`

Beyond pattern matching, a decrease can also be established by selecting a field of a value. The helper `peelSelects` resolves a chain of field-access expressions back to an original symbol and determines whether a decrease occurred along the way. A field selection is considered to decrease the size of the value when the selected field's name corresponds to a constructor parameter of a `case class`, or when the field's method is annotated with `@assumeDecreases`.

4.1.3 Verifying Polymorphic Method Calls

Object-oriented languages like Scala heavily utilize subtype polymorphism, where a method called on a value of a static type (such as a sealed trait or abstract class) is dynamically dispatched at runtime to a concrete implementation. This introduces a verification challenge not typically present in proof assistants like Rocq or Lean. Because our checker operates at compile time, it cannot definitively know which concrete method will be invoked at runtime.

In the presence of dynamic dispatch, the checker must over-approximate the possible runtime behaviors. When a method call is encountered on a polymorphic type, the checker leverages Scala's `sealed` keyword, which restricts the set of possible subtypes to a known, finite set. The `methodOverrides` routine enumerates all sealed strict descendants of the static type and resolves the method name within each descendant's type. If any overrides are found, the checker verifies each one independently. A polymorphic call is accepted only if *every* possible concrete implementation it could dispatch to is proven to terminate. By exhaustively checking the sealed hierarchy, the checker guarantees that whichever method is dynamically dispatched at runtime is safe. If even a single overriding method fails the termination check or if the hierarchy is not sealed, meaning an unknown subclass could be loaded at runtime the checker conservatively rejects the call.

4.1.4 Well-Foundedness

A structural decrease is only a valid termination argument if the type being decreased is well-founded, that is, it admits no infinite descending chain of sub-values. The checker enforces this

via `typeWellFounded`. A sealed type is well-founded when all of its children are well-founded. A case object (a leaf of an ADT) is trivially well-founded. A case class is well-founded when all of its fields are stable members that correspond directly to constructor parameters. If the type is not well-founded, the checker emits a warning and considers the value of size `Unknown`.

4.1.5 Lexicographic Ordering

When checking a recursive call, the method `areSmaller` compares the list of arguments against the list of parameters of the recursive method in order. It calls `isLexicoDecreasing`, which steps through the argument–parameter pairs. If the first pair is `Smaller`, the check succeeds provided the type is well-founded. If it is `Same`, the check proceeds to the next pair. If it is `Unknown`, the check fails. The ordering of the pairs can be controlled by the `@decreases` annotation, which specifies an ordering of parameters to use as the measure, overriding the default left-to-right order.

4.2 Mutual Recursion

The checker handles mutual recursion by inlineing called methods. A `callStack` tracks the methods currently being analyzed. When a call to a method is encountered, the checker first looks it up in `callStack`. If it is already present, the call is recursive (possibly mutual) and `areSmaller` is invoked to verify the decrease. If the method is not yet on the stack, `traverseCalled` is invoked, which pushes the callee onto the stack, maps the actual arguments to the callee's parameters in `valMap`, traverses the callee's body, and then restores the stack and map. This allows the checker to follow chains of mutual recursion of arbitrary depth.

4.3 While Loops

While loops are treated separately from recursive calls. Since they rely on mutable variable. The checker performs two traversals. In the first traversal, call checking is disabled and `varMap` is reset to empty. The purpose of this traversal is to determine whether there exists a variable that is assigned a strictly smaller value (with respect to itself) inside the loop body. If such a variable exists the loop terminates. Note that since well-founded types cannot decrease indefinitely, we have two possible cases. Either the loop condition becomes false, hence, the loop terminates. Or we hit a leaf of the ADT and the assignment to a strictly smaller value results in an exception being thrown (this is the case for e.g. `Nil.tail`) or the `@assumeDecreases` escape hatch was wrongly used. The checker then proceeds to a second traversal with call checking re-enabled,

and the original `varMap` restored so that any recursive calls inside the loop body are themselves verified to be terminating.

4.3.1 `varMap` Updates

An `Assign` node records `Smaller` in `varMap` if `areSmaller` can confirm the right-hand side is smaller than the current value of the variable, and `Unknown` otherwise.

For branching and looping constructs, the checker explores each branch independently and then merges the results where the worst-case size is kept. Concretely, when traversing an `If` expression or `Match` cases, we compute one `varMap` for each branch, then, for each variable in the union of all maps, it is considered `Unknown` if `Unknown` in any branch, `Smaller` if `Smaller` in all branches and `Same` otherwise. Then the original `varMap` is updated with the new information from the branching construct. For `WhileDo`, the body may never be executed, therefore, we merge the original `varMap` with a `varMap` computed by traversing the body of the loop.

Chapter 5

Evaluation and Limitations

5.1 Performance Evaluation

To evaluate the compile-time overhead introduced by the checker, benchmarks were run on a file containing approximately 30 functions. These benchmarks were conducted on a personal computer while no other processes were running to ensure the accuracy and consistency of the measurements. The compilation was measured across three scenarios: using no flags to completely disactivate the phase, using only the `-Yretain-trees` flag to measure the overhead of keeping the AST in memory, and using both `-Yretain-trees` and `-Ycheck-termination` to fully activate the checker. The benchmarks show that the checker has little overhead. Retaining

Configuration	Average Time (ms/op)	Error Margin
Baseline	84.246	± 0.423
`-Yretain-trees`	85.301	± 1.029
Checker enabled	85.769	± 0.614

Table 5.1: Compile-time overhead of the termination checker (averaged over 20 iterations).

the trees introduces an overhead of 1.06 ms over the baseline. Fully activating the checker adds an additional 0.47 ms. Overall 1.8% increase in compilation time is negligible. This confirms that the goal of being lightweight is achieved.

5.2 What the Checker Can Verify

5.2.1 Simple Structural Recursion

The most direct case is a recursive function that pattern-matches on an ADT and recurses on a field extracted by that match.

```
sealed abstract class Nat
case object Zero extends Nat
case class Succ(val n: Nat) extends Nat

@terminates
def f(n: Nat): Boolean =
  n match
    case Zero => false
    case Succ(pn) => f(pn)
```

When the checker encounters the `match` on `n`, it identifies a synthetic `unapply` method, and records `pn` as `Smaller` than `n` in `valMap`. At the recursive call `f(pn)`, `areSmaller` determines that the argument `pn` is `Smaller` than the parameter `n`, and `typeWellFounded` confirms that `Nat` is a well-founded type. The recursive call is therefore accepted.

5.2.2 Lexicographic Decrease and `@decreases`

Some functions require lexicographic ordering. For the Ackermann function the parameter `n` decreases in some calls, while in others `n` stays the same but `m` decreases.

```
@terminates
def ackermann(n: Nat, m: Nat): Nat =
  n match
    case Zero => Succ(m)
    case Succ(pn) =>
      m match
        case Zero => ackermann①(pn, Succ(Zero))
        case Succ(pm) => ackermann②(pn, ackermann③(n, pm))
```

For ① and ②, the checker sees `pn` is `Smaller` than `n` and immediately accepts. For ③, `n` is `Same`, so `isLexicoDecreasing` advances to the second position and finds `pm` is `Smaller` than `m`.

All three recursive calls are therefore accepted. One limitation we see here, is that we must match on one value at a time and matching over tuples is not supported.

When the decreasing argument is not the first, `@decreases` can be used to tell the checker to use a different ordering:

```
@terminates
def f(logger: List[Nat], n: Nat): (Nat, List[Nat]) @decreases(n) =
  n match
    case Zero => (Zero, Zero :: logger)
    case Succ(pn) => f(n :: logger, pn)
```

Without `@decreases(n)`, the checker would compare `n :: logger` to `logger` first, find the size unknown, and reject the call. With the annotation, it checks `n` first, finds `pn` is `Smaller`, and the call is accepted. Note that the annotation is on the output type to allow the use of function parameter symbols in the annotation parameters.

5.2.3 Mutual Recursion

```
@terminates
def isEven(n: Nat): Boolean =
  n match
    case Zero => true
    case Succ(pn) => isOdd(pn)
```

```
@terminates
def isOdd(m: Nat): Boolean =
  m match
    case Zero => false
    case Succ(pm) => isEven(pm)
```

When `isEven` calls `isOdd`, the checker pushes `isOdd` onto `callStack` and maps argument `pn` to parameter `m` of `isOdd`. Inside `isOdd`'s body it finds the call `isEven(pm)`. At this point `isEven` is already on `callStack`, so the call is flagged as (mutually) recursive and `areSmaller` is called. The argument passed to `isEven` is the `pm`, and we have the following chain of size inequalities asserting `pm` is `Smaller` than `n`: `pm` < `m` = `pn` < `n`. Therefore, both functions are shown to be terminating.

5.2.4 Higher-Order Functions

The checker can follow higher-order calls when the closure is available.

```
sealed trait MyList[+A]:
  def foldRight[B](z: B)(op: (A, B) => B): B = this match
      case MyNil => z
      case MyCons(hd, tl) => op(hd, tl.foldRight(z)(op))
case object MyNil extends MyList[Nothing]
case class MyCons[+A](val hd: A, val tl: MyList[A]) extends MyList[A]

sealed trait Expr
case class Const(c: Int) extends Expr
case class Add(args: MyList[Expr]) extends Expr

@terminates
def addAll(e: Expr): Int =
  e match
    case Const(c) => c
    case Add(args) => args.foldRight(0)((l, acc) => addAll(l) + acc)
```

When `foldRight` is inlined and the checker encounters the application of `op`, `getMethodSymbol` resolves it back to the anonymous function `(l, acc) => addAll(l) + acc`. Inside that definition, the recursive call `addAll(l)` uses `l`, which is `Smaller` than `e` by the following chain of size inequalities: `l` = `hd` < `this` = `args` < `e`. Furthermore, `foldRight` terminates as its `this` arguments decreases. We can view this as mutual recursion: `addAll` calls `foldRight` and `foldRight` calls `addAll` through `op`. Note, Stainless is currently unable to prove termination in this case and provides a counter-example which is false.

5.2.5 While Loops

A while loop terminates if some mutable variable is unconditionally assigned a strictly smaller value in every iteration.

```
@terminates
def f(a: List[Int]): Int =
  var res = 0
  var l = a
  while (!l.isEmpty) {
    l = l.tail
    res += f(l)
  }
  res
```

In the first traversal, the checker processes the assignment `l = l.tail`, `peelSelects` resolves all possible `tail` methods/values and checks whether they are constructor parameters or annotated with `@assumeDecreases`. In this case we have annotated the standard library `tail` methods of `List`. Since a decreasing variable exists, the loop is accepted. The second traversal then verifies that the recursive calls inside the loop body also terminate.

5.3 What the Checker Cannot Verify

5.3.1 Semantic Termination Arguments

Because sizes are inferred syntactically, the checker rejects any function whose termination depends on semantic properties.

```
@terminates
def f(a: List[Int]): Int =
  a match
    case Nil => 0
    case _ => g(a)
def g(a: List[Int]): Int =
  a match
    case Nil => f(a) // error
    case hd :: tl => hd + f(tl)
```

Here, `f` terminates, however, the recursive call `f(a)` in `g` is rejected by the checker since its unaware that the call `g(a)` cannot be reached as `a = Nil`.

```
@terminates
def h(a: List[Int]): Int =
  var res = 0
  var l = a
  while (!l.isEmpty) { // error
    res += l.head
    if true then l = l.tail
  }
  res
```

Similarly, here `l` is only assigned `l.tail` in one branch of the `if`. After merging the two branches, `varMap` records `l` as `Same` (since the else branch does not decrease it), and the

checker rejects the loop, even though the condition `true` means the else branch is dead code. The checker is purely syntactic and does not evaluate conditions.

5.3.2 Cross-Parameter Size Tracking

The checker does not support swapping decreasing arguments.

```
@terminates
def i(n: Nat, m: Nat): Nat =
  n match
    case Zero => Zero
    case Succ(pn) =>
      m match
        case Zero => Zero
        case Succ(pm) => i(pm, pn) // error
```

The function `i` terminates, it can be seen by unrolling the recursive call. However, when the recursive call `i(pm, pn)` is found, the method `areSmaller` compares `pm` with `n` which results in `Unknown`. This highlights a fundamental limitation of the checker's structural analysis, as it verifies sizes strictly on a per-parameter basis. Techniques such as size-change termination, or the SMT-driven bounds analysis used by Stainless, can handle these cross-parameter cases but come with a higher computational overhead.

Chapter 6

Correctness

In this chapter we give a correctness sketch for the termination checker. We use the term *termination* to mean that a function either returns a value or throws an exception. Infinite loop or recursion are the only failure modes the checker is concerned with. Under this definition, a thrown exception counts as a form of termination, because the function does eventually stop executing. Correctness in this context means *soundness* under any assumptions made by escape hatches, i.e., if the checker accepts a method, then that method terminates in the sense defined above, conditional on all assertions made.

6.1 Simple Recursion

Consider a method `f` with parameters $p_1, \ldots, p_k$ that has been accepted by the checker. By construction, every call within `f`'s body that is detected as recursive must pass `areSmaller`, which asserts that the argument tuple $(a_1, \ldots, a_k)$ is lexicographically smaller than $(p_1, \ldots, p_k)$ with respect to the structural order of each type.

More precisely, `isLexicoDecreasing` finds an index i such that:

$$a_1 = p_1, \ldots, a_{i-1} = p_{i-1}, \quad a_i < p_i,$$

where $<$ denotes strict structural sub-value. The size of each argument is determined syntactically by `compareSize`, which follows the `valMap` chain back to a parameter of the method. A value recorded as `Smaller` means it was extracted from a constructor pattern or from a field access on a `case class`, guaranteeing that it is a proper sub-value of the original.

A structural sub-value relationship is only a valid termination argument when the type

in question is *well-founded*, i.e., it admits no infinite descending chain of sub-values. The checker enforces this via `typeWellFounded`. A `sealed` type is well-founded if every child is well-founded, case objects are trivially well-founded (they are leaves), and a case class is well-founded when all of its fields are stable members corresponding directly to constructor parameters. Because every recursive call strictly decreases the argument tuple in this well-founded order, there can be no infinite sequence of recursive calls, so `f` terminates.

6.2 Mutual Recursion

The checker handles mutual recursion by inlining method calls. When `f` calls a method `g` that is not yet on the `callStack`, `g`'s body is traversed with `g` pushed onto the stack and the actual arguments mapped to `g`'s parameters in `valMap`. Any call inside `g`'s body that is already on the stack (whether to `g` or back to `f`) is treated as a recursive call and must satisfy `areSmaller` with respect to the that method's parameters.

The invariant is that the `callStack` represents the current chain of inlined calls. When a method already on the stack is called, the checker checks that the arguments to that method's original parameters are strictly smaller. Since `valMap` propagates size information through argument-to-parameter mappings as each callee is pushed, size inequalities can be composed across an arbitrary chain of mutual calls.

For example, in the mutual recursion between `isEven` and `isOdd` over the `Nat` ADT in subsection 5.2.3, when the checker encounters the call `isEven(pm)` inside `isOdd`, it traces the chain `pm` < `m` = `pn` < `n` through `valMap`, establishing that the argument to `isEven` is `Smaller` than `isEven`'s original parameter. Because `Nat` is well-founded, the call is accepted and both functions terminate.

More generally, suppose the inlined call chain is $f_0 \to f_1 \to \cdots \to f_n \to f_k$ for some $k \leq n$. The checker requires that the arguments passed to f_k are strictly smaller than f_k's parameters in the well-founded order. Since `valMap` records the accumulated size relationships through every parameter mapping along the chain, a `Smaller` assertion implies that each iteration of the mutual cycle strictly decreases the relevant argument. Again, by well-foundedness, this is possible finitely many times.

6.3 While Loops

A while loop is analyzed by two traversals of its body. In the first traversal, recursive call checking is disabled and `varMap` is initialized to empty. The checker looks for a mutable

variable `v` such that every execution path through the loop body assigns `v` a value that is `Smaller` (i.e., a structural sub-value of `v`'s current value). The merging rules for `varMap` over branches ensure that if any branch leaves `v` at `Same` or `Unknown`, it is not counted as a decreasing variable. Only a variable that decreases in *all* branches qualifies and if no such variable exists the checker reports an error.

Once a decreasing variable `v` is identified, termination of the loop follows from well-foundedness of `v`'s type. Because `v` is strictly smaller after every iteration, and the type of `v` is well-founded (no infinite descending chains exist), the loop must eventually reach a point where the assignment `v = v.someField` would dereference a leaf value (such as `Nil.tail`), at which point an exception is thrown (which counts as termination) or the loop condition becomes false before that point is reached. Both are valid terminations under our definition.

The second traversal then verifies, that any recursive calls inside the loop body also terminate.

6.4 Higher-Order Functions

When a call to a higher-order function is encountered, the checker inlines the callee's body just as for ordinary calls. Inside that body, if the higher-order argument (e.g., an `op` parameter of type `(A, B) => B`) is applied, `getMethodSymbol` resolves the application back to the concrete closure that was passed at the call site. The closure's body is then traversed under the same call stack and size maps. This means that a recursive call made inside a closure passed to a higher-order function is treated exactly like a recursive call to an ordinary function.

As illustrated by the `addAll`/`foldRight` example in subsection 5.2.4, the checker can establish the chain `hd` < `this` = `args` < `e` through inlining of `foldRight` and resolution of `op` to the closure. The correctness argument reduces to the mutual recursion case.

6.5 Soundness Under Escape Hatches

In all cases, the checker does not verify the assertions made by any escape hatches, and it is the developer's responsibility to ensure they are correct. Under correct use of escape hatches, the conditional soundness guarantee is as strong as the unconditional one.

Chapter 7

Discussion

7.1 Annotations Added to the Standard Library

Integrating the checker with idiomatic Scala code requires annotating parts of the standard library that the checker cannot verify automatically. Two categories of annotations are necessary.

Adding `@assumeWellFounded` on well-founded types that use mutable variables under the hood. The `List[+A]` type is declared `sealed abstract class` with two subtypes: `Nil` a `case object` and `::` a `case class`. This almost satisfies the checker's well-foundedness, however, `::` declares its `next` field as `private[scala] var`, making it a mutable field rather than a stable member, therefore, failing the check. Other types in the standard library may benefit from similar semantic assertions that are syntactically false.

Adding `@assumeDecreases` on `head` and `tail`. Since they are defined as methods, the checker cannot infer a decrease automatically, adding the annotation tells the checker that calling `.tail` on a `List` value yields a strictly smaller value. The same annotations are added on `Nil`, even though those methods throw exceptions rather than returning a smaller value. This is correct under the termination definition where exceptions are a form of termination. Annotating them allows the checker to accept calls on these methods on any `List` regardless of whether they are empty or not which the checker cannot infer.

Furthermore, methods like `+`, `-`, `<`, etc. on `Int`, `Float`, and other primitive types have their implementations in compiled bytecode. Their `defTree` is empty, so the checker emits warnings as their bodies are available for analysis. To suppress these warnings `@assumeTerminates` annotations where added to these operators.

7.2 Lexicographic Ordering over the Size-Change Principle

The global size-change principle [LJB01] was considered as an alternative to the per-call lexicographic check. It would handle the cross-parameter swapping case and some mutual recursion patterns that the current checker cannot. However, the size-change principle has been shown to be PSPACE-complete. For a compiler phase that runs on every build, a PSPACE algorithm over the full call graph is not acceptable. Furthermore, when the size-change check fails, the failure is a global property of the call graph rather than a property of a specific call site. The per-call lexicographic check, by contrast, fails at a precise source location with a precise argument, making it straightforward to guide the developer toward a fix or an appropriate escape hatch.

7.3 Limitations

Functions that terminate because an integer decreases toward a bound, rather than a data structure shrinks, cannot be verified. This covers a significant class of imperative algorithms. The most natural extension would be to allows the `@decreases` annotation to accept a well-founded integer relation, checked at each loop iteration or recursive call. This would not necessarily require an SMT solver and could be implemented within the same syntactic framework.

When a method is called on a value whose static type is an unsealed trait or abstract class, the checker cannot enumerate the possible concrete implementations and conservatively rejects the call. In practice, this is most likely to arise with third-party library types that the checker has no visibility into. The correct response in those cases is to wrap the call in `uncheckedTermination` or to annotate the callee with `@assumeTerminates`.

Chapter 8

Conclusion

This report has presented the design, implementation, and evaluation of an opt-in termination checker for the Scala 3 compiler. The checker operates as a `MiniPhase` on the typed AST, performs structural decrease analysis over Algebraic Data Types, and handles recursive functions, mutual recursion, and while loops. It is activated by the `-Ycheck-termination` compiler flag and applies only to methods annotated with `@terminates`, making it fully backward-compatible with existing Scala projects.

The implementation demonstrates that a useful termination guarantee can be provided within a general-purpose compiler without relying on SMT solvers, rewriting engines, or external tools. For the patterns that Scala developers most commonly encounter: structural recursion over inductively defined data, mutual recursion between methods, and loops driven by a structurally decreasing variable, the checker provides a lightweight mechanism that is immediately available at compile time.

The principal limitations are the absence of cross-parameter size tracking, and the lack of numeric decrease analysis. Each of these represents a well-understood problem with known techniques for improvement, and addressing them in future work would bring the checker closer to the expressiveness of more heavyweight tools.

In sum, the checker occupies a practical middle ground in the landscape of termination verification. More accessible than a proof assistants while providing a verification framework for safety-critical code, library foundations, and any context where termination is a correctness requirement, the `@terminates` annotation provides a low-overhead way to document and enforce that guarantee.

Bibliography

[BKO26] Matt Bovel, Viktor Kunčak, and Martin Odersky. *First-Class Refinement Types for Scala*. 2026. arXiv: 2605.08369 [cs.PL]. URL: https://arxiv.org/abs/2605.08369.

[DM79] N DERSHOWITZ and Z MANNA. "Proving Termination with Multiset Orderings". English. In: *Communications of the ACM* 22.8 (Aug. 1979), pp. 465–476. ISSN: 0001-0782. DOI: 10.1145/359138.359142.

[EHG25] Frank Emrich, Jera Hensel, and Jürgen Giesl. *AProVE: Modular Termination Analysis of Memory-Manipulating C Programs*. 2025. arXiv: 2302.02382 [cs.LO]. URL: https://arxiv.org/abs/2302.02382.

[Gie+11] Jürgen Giesl, Matthias Raffelsieper, Peter Schneider-Kamp, Stephan Swiderski, and René Thiemann. "Automated termination proofs for haskell by term rewriting". In: *ACM Trans. Program. Lang. Syst.* 33.2 (Feb. 2011). ISSN: 0164-0925. DOI: 10.1145/1890028.1890030. URL: https://doi.org/10.1145/1890028.1890030.

[HVK19] Jad Hamza, Nicolas Voirol, and Viktor Kuncak. "System FR as Foundations for Stainless". In: *CoRR* abs/1904.03482 (2019). arXiv: 1904.03482. URL: http://arxiv.org/abs/1904.03482.

[JUN+18] RALF JUNG, ROBBERT KREBBERS, JACQUES-HENRI JOURDAN, ALEŠ BIZJAK, LARS BIRKEDAL, and DEREK DREYER. "Iris from the ground up: A modular foundation for higher-order concurrent separation logic". In: *Journal of Functional Programming* 28 (2018), e20. DOI: 10.1017/S0956796818000151.

[LJB01] Chin Lee, Neil Jones, and Amir Ben-Amram. "The Size-Change Principle for Program Termination". In: *ACM SIGPLAN Notices* 36 (Jan. 2001). DOI: 10.1145/360204.360210.

[MU21] Leonardo de Moura and Sebastian Ullrich. "The Lean 4 Theorem Prover and Programming Language". In: *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635. ISBN: 978-3-030-79875-8. DOI: 10.1007/978-3-030-79876-5_37. URL: https://doi.org/10.1007/978-3-030-79876-5_37.

[Nor07] Ulf Norell. "Towards a practical programming language based on dependent type theory". PhD thesis. SE-412 96 Göteborg, Sweden: Department of Computer Science and Engineering, Chalmers University of Technology, Sept. 2007.

[PLO17] Dmitry Petrashko, Ondřej Lhoták, and Martin Odersky. "Miniphases: compilation using modular and efficient tree transformations". In: *SIGPLAN Not.* 52.6 (June 2017), pp. 201–216. ISSN: 0362-1340. DOI: 10.1145/3140587.3062346. URL: https://doi.org/10.1145/3140587.3062346.

[PNW19] Lawrence C. Paulson, Tobias Nipkow, and Makarius Wenzel. "From LCF to Isabelle/HOL". In: *CoRR* abs/1907.02836 (2019). arXiv: 1907.02836. URL: http://arxiv.org/abs/1907.02836.

[Tea25] The Rocq Development Team. *The Rocq Prover*. Version 9.1.0. Sept. 2025. DOI: 10.5281/zenodo.17473943. URL: https://doi.org/10.5281/zenodo.17473943.

[TF25] Yee-Jian Tan and Yannick Forster. *Towards Formalising the Guard Checker of Coq*. Tech. rep. Ecole polytechnique ; Inria - Paris, Mar. 2025, p. 1. URL: https://inria.hal.science/hal-04983786.

[Tur37] Alan M. Turing. "On Computable Numbers, with an Application to the Entscheidungsproblem". In: *Proceedings of the London Mathematical Society* s2-42.1 (1937), pp. 230–265. DOI: 10.1112/plms/s2-42.1.230.

[Win93] Glynn Winskel. *The formal semantics of programming languages: an introduction*. Cambridge, MA, USA: MIT Press, 1993. ISBN: 0262231697.